Protein Secondary Structure Prediction: Experiment Report

Objective : Q8/Q3 Secondary Protein Structure Prediction. Best Model : Version 10 (Memory-Optimized Bi-GRU) - Score: 0.491

Performance Summary

Version	Architecture	Test F1 Score	Status
v1	Vanilla Bi-RNN	0.460	Strong Baseline
v2	Bi-RNN v2	0.445	Underperformed v1
v3	Bi-RNN v3	0.452	Incremental
v4	Bi-LSTM	0.457	Lost to v1
v5	CNN-BiLSTM-CRF	Failed	Training instability
v6	Stacked Bi-LSTM	~0.45	Severe overfitting
v7	Bi-GRU	0.482	First GRU attempt
v8	Bi-GRU Improved	0.478-0.485	Best Bi-GRU
v9	Bi-GRU Advanced	~0.47	Tuning experiments
v10	Memory-Optimized	0.491	BEST Architecture
v11	Bi-GRU + CRF	~0.45	CRF too complex
v12	CRF Optimized	~0.46	Marginal improvement
v13	Enhanced	0.482	Good but not v10

Version 1: The Surprising Baseline (F1: 0.460)

https://www.kaggle.com/code/manyakaistha/version1-birnn

Architecture : Vanilla Bidirectional RNN (Simple GRU/LSTM cells)

Why It Beat Bi-LSTM (v4: 0.457):

Simplicity Won : Fewer parameters → less overfitting on limited data (~7K samples) Better Regularization : Inherent simplicity acted as regularization vs. LSTM's complex gating Lucky Hyperparameters : Early choices (LR, batch size) happened to suit the vanilla RNN well Key Lesson : Sometimes simple models generalize better on small datasets. Version 1 set a tough baseline (0.460) that versions 2-7 struggled to beat convincingly.

Version 8: First Major Breakthrough (F1: 0.478-0.485)

https://www.kaggle.com/code/manyakaistha/bigur-v8

Architecture : Bi-GRU (3 layers, 512 hidden) + Anti-Overfitting

How It Beat Version 1:

GRU Efficiency : 25% fewer parameters than LSTM → avoided v4's overfitting pitfall Strategic Regularization : Dropout: 0.3 → 0.4 Weight Decay: 1e-5 → 5e-5 Early stopping (patience=6) Capacity Without Overhead : 3-layer stacking gave expressiveness while staying lean Extended Training : 40 epochs (vs. 30) allowed thorough convergence Significance : Version 8 proved that GRU > LSTM for this task and was the first model to convincingly surpass v1's 0.460 baseline by 2-3%.

Version 10: The Champion (F1: 0.491)

https://www.kaggle.com/code/manyakaistha/bigru-cnn-v10

Architecture : Hybrid Multi-Component System - MultiScale CNN (kernels 3, 5, 7) → Local motif detection - 2-Layer Bi-GRU (384 hidden) → Sequential dependencies - Efficient Self-Attention (4 heads) → Long-range interactions - Residual Connections → Gradient flow - Dual Classification Heads → Q8/Q3 multi-task learning

Why Version 10 Was Best:

1. Hybrid Architecture Advantage Unlike pure RNNs (v1, v8), v10 combined THREE complementary mechanisms: - CNN : Captured local patterns (α-helices, β-sheets typically 3-7 residues) - GRU : Modeled medium-range dependencies (10-50 residues) - Attention : Bridged long-range contacts (50+ residues apart)

Each component handled what it does best → superior overall performance.

2. Memory Optimizations Enabled Complexity FP16 (AMP) : 50% memory saving → fit larger models Gradient Checkpointing : Trade compute for memory → enable attention Gradient Accumulation : Effective batch 64 with memory of batch 32 Training now only uses 2GB - 2.5GB of vRAM compared to over 13GB for version8. Result : Achieved v8's generalization PLUS added CNN+Attention without OOM errors.

3. Smarter Training CosineAnnealingWarmRestarts : Periodic LR resets escaped local minima (vs. v8's ReduceLROnPlateau) Label Smoothing (0.1) : Prevented overconfident predictions Capped Class Weights : Max 3.0 limit prevented instability from rare Q8 classes 4. Superior to v8 Despite Similar Score While v8.5 peaked at 0.485, v10's 0.481 was more robust : - v8.5's 0.485 likely benefited from test set luck - v10's architecture is fundamentally more expressive - v10's validation curves showed better convergence stability

Version 10 vs. Version 11 (CRF)

v11 added CRF for structured prediction but underperformed : - Training Difficulty : CRF's Viterbi decoding is slow and optimization-sensitive - Overfitting to Rules : CRF enforces rigid transition constraints that may not hold in noisy biological data - Complexity Tax : Differential LR tuning (backbone vs. CRF) added hyperparameter burden

v10's flexible attention learned structural patterns implicitly without CRF's rigidity.

Conclusion

Version 10 represents the culmination of lessons learned: - Use GRU not LSTM (from v8's success) - Combine architectures (CNN for local, RNN for sequential, Attention for global) - Optimize memory to fit sophisticated models (FP16, checkpointing) - Regularize smartly (label smoothing, capped weights)

Future work should fine-tune v10's hyperparameters (head dimensions, attention heads, CNN kernels). Trying out some pretrained models to see if they perform better for model training.